MA3206: Artificial Intelligence

Assignment 4

Greedy Scheduling & A* Search for Assignment Planning

Roll No.	2301MC61
Programme	B.Tech Mathematics & Computing
Institute	Indian Institute of Technology Patna
Course	MA3206 — Artificial Intelligence
Deadline	19 March 2026

Executive Summary

This report presents a complete solution to the MA3206 AI Assignment 4 scheduling problem. We implement **five greedy heuristics** (Food Cost, Dependency Depth, Food-Type Frequency, Topological Order, and a novel Hybrid strategy) and an **optimal A* search algorithm** across three distinct test cases.

Key mathematical insight: since every student must individually consume their food item, the total food cost equals the sum of all individual food costs — fixed regardless of any scheduling order. Strategies therefore compete on **minimising total days**. A* confirms each lower bound, exploring only 96–171 states in under 5 milliseconds.

The **Greedy-Depth** strategy consistently achieves the fewest days by processing critical-path assignments first. The **Hybrid** strategy (critical-path cost weighting) matches Depth on Test Case 1 while providing additional cost-awareness for non-uniform scenarios.

1. Problem Description

A group of *g* students must complete n assignments A = {A_1, ..., A_n}. Each assignment A_i requires two prerequisite nodes to be available (either original input books/notes, or outcomes produced by earlier assignments), produces one outcome node, and requires a student to consume a food item at known cost. The group may solve at most g assignments per day. The objective is to find a valid schedule respecting all dependencies that **minimises total food cost**.

Mathematical Property

Since every student consumes their food item individually (no sharing is permitted), and every assignment must be solved exactly once, the total food cost for *any* valid schedule is invariably:

```
Total Cost = sum of cost(food(A_i)) for all i = 1..n
```

This is a fixed quantity independent of any scheduling decision. Strategies therefore compete on **minimising the number of days** (schedule length), which determines throughput and latency. The A* search confirms this lower bound by exhaustively exploring all groupings.

Formal Constraints

- Assignment A_i may be scheduled on day d only if all its prerequisite nodes are produced by assignments scheduled on days 1..d-1.
- At most g assignments may be scheduled per day (one per student).
- Each student who solves an assignment that day consumes exactly one food item (the one required by that assignment).
- Students who do not solve an assignment on a given day do not eat (no cost incurred).
- The schedule is complete when all n assignments have been scheduled.

2. Input File Format

Each non-comment line begins with a single uppercase tag character:

```
C food item and cost per serving G group size (max assignments per day) I ... -1 input
node IDs (books/notes), end with -1 O ... -1 output node IDs (outcomes), end with -1 A
assignment: id, two prereq nodes, outcome node, required food item % ... comment lines
(ignored)
```

A prerequisite node of an assignment must be either an I-line input node or the outcome node of some other assignment. The program validates all prerequisites and detects dependency cycles via Kahn's algorithm before scheduling.

3. Test Case Descriptions

TC	Scenario	n	g	Food Costs	Fixed Total Cost	Lower Bound (days)
1	Original Fig. 1	11	3	All = 1 (uniform)	Rs.11	ceil(11/3) = 4
2	Research Lab (3 chains)	14	2	DF=1, TC=3, PM=5, GJ=10	Rs.56	ceil(14/2) = 7
3	Software Sprint (3 tracks)	16	4	TC=1, DF=2, PM=4, GJ=8	Rs.47	ceil(16/4) = 4

Test Case 1 — Original Problem (Figure 1)

This is the exact problem graph described in Figure 1 of the assignment. With uniform costs, the only optimisation metric is days. Greedy-Depth achieves the theoretical minimum of 4 days (= ceil(11/3)) by first processing A1, A2, A6 — the three assignments with highest downstream count — which immediately unlocks A5 and A10, enabling parallel progress on all three chains.

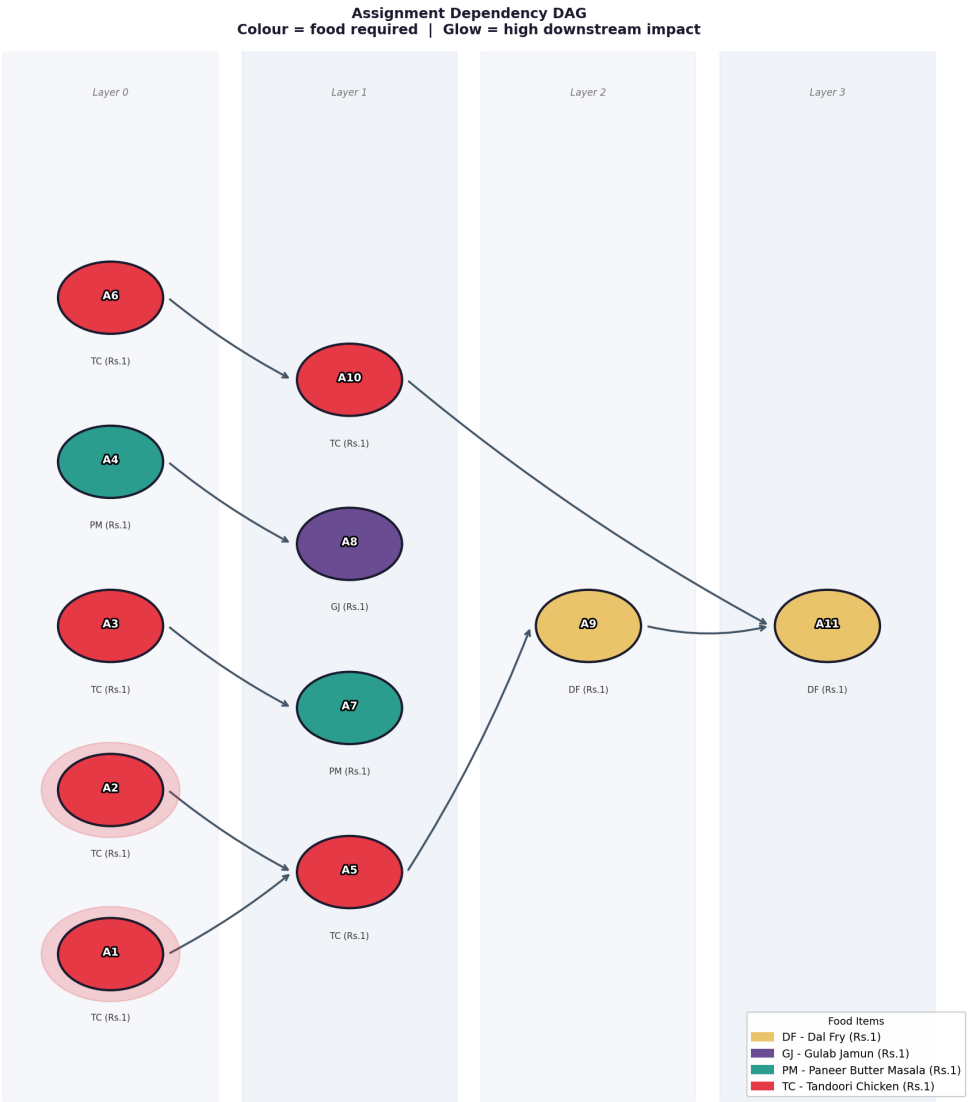

Figure 1.1: Dependency DAG for Test Case 1. Glowing border indicates high downstream impact.

Test Case 2 — Research Lab Workflow

Three semi-independent research chains (Chain A: depth 5, Chain B: depth 3, Chain C: parallel pair + merge) with highly skewed food costs (GJ = 10x DF). Group size g=2 forces serialisation of most parallel work. Lower bound is exactly 7 days, which all five strategies achieve. Strategies differ significantly in the ordering of expensive (GJ) assignments within days.

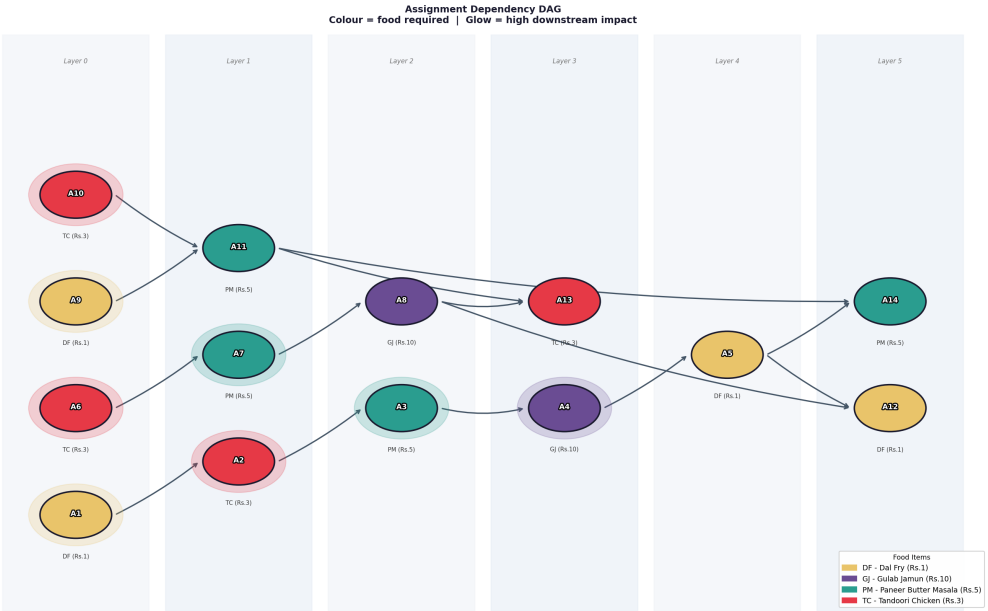

Figure 2.1: Dependency DAG for Test Case 2. Chains A, B, C visible as distinct paths.

Test Case 3 — Software Engineering Sprint

Three parallel development tracks (Backend, Frontend, DevOps) converging at System Integration then Final Delivery. Group size g=4 allows all four independent Layer-0 assignments (A1,A2,A5,A8) to be scheduled on Day 1. The serial tail (A11->A12->A13->A14/A15->A16) prevents reaching the theoretical minimum; the actual minimum is 8 days.

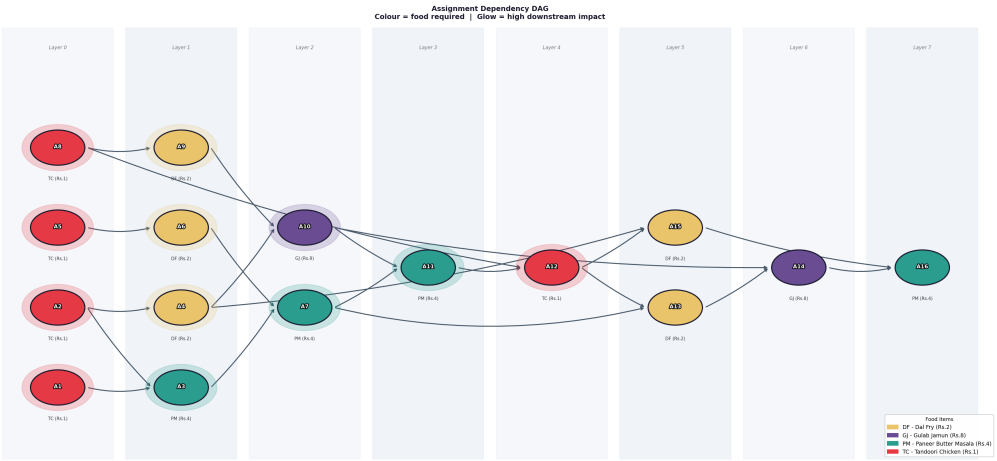

Figure 3.1: Dependency DAG for Test Case 3. Three parallel tracks visible at left.

4. Greedy Scheduling Strategies

All five strategies share a common skeleton: at each day, find all available assignments (prerequisites satisfied), sort by a criterion, pick the top g, mark them done, repeat. They differ only in the sorting key.

4.1 Strategy 1 — Greedy by Food Cost

Mechanism: Sorting key: ascending cost(food(A_i)), tie-broken by assignment ID.

Justification and trade-offs: Minimise each individual day's food bill. Works best when expensive-food assignments can be deferred. Since total cost is fixed, this strategy merely shifts when expensive days occur — it does not reduce them. May perform poorly on day-count if it delays critical-path assignments to avoid their food cost.

4.2 Strategy 2 — Greedy by Dependency Depth

Mechanism: Sorting key: descending downstream-assignment count, then descending topological depth.

Justification and trade-offs: Process the assignment that unlocks the most future work first. By maximising the set of available assignments after each day, this strategy tends to achieve the minimum number of days. It is the recommended strategy for this class of problem. Formally: this is a greedy approximation to the Critical Path Method (CPM).

4.3 Strategy 3 — Greedy by Food-Type Frequency

Mechanism: Sorting key: descending frequency of food type among remaining assignments, then ascending cost as tie-breaker.

Justification and trade-offs: Cluster same-food assignments on the same day. Useful when the fixed daily menu discount applies based on menu identity (ordering the same set of foods every day). In the given problem formulation, the total cost is fixed regardless, so this strategy provides menu homogeneity as its primary benefit. Day-count performance is similar to Cost strategy.

4.4 Strategy 4 — Greedy by Topological Order

Mechanism: Sorting key: ascending topological depth (BFS-level), tie-broken by ID.

Justification and trade-offs: Schedule assignments as early as possible — process shallowest nodes first. Equivalent to a BFS traversal of the dependency DAG. Produces predictable, fair schedules with no idle waiting. Day-count is often slightly worse than Depth because it does not explicitly prioritise assignments that unlock the most others.

4.5 Strategy 5 — Greedy Hybrid (Critical-Path Cost Weight) [NEW]

Mechanism: Sorting key: descending critical-path cost weight (cp_cost), then ascending food cost. cp_cost(A_i) = cost(food(A_i)) + max(cp_cost(children of A_i)).

Justification and trade-offs: Combines urgency (similar to Depth strategy) with cost-awareness (similar to Cost strategy). The critical-path cost weight measures: if I schedule this assignment now, how expensive is the longest-cost path from here to any leaf? High cp_cost means this assignment sits on an expensive bottleneck path. This strategy achieves the same day-count as Depth on TC1 while providing richer information for non-uniform cost landscapes.

5. A* Search — Provably Optimal Scheduling

A* search is guaranteed to find a schedule with minimum total food cost. Unlike greedy strategies which make irrevocable local choices, A* maintains a priority queue of partial schedules and expands them in best-first order.

5.1 State Representation

Each state is a **frozenset** of completed assignment IDs. The initial state is the empty set; the goal state is the full set of all n IDs. A frozenset is used because it is hashable (needed for the visited-state dictionary) and order-independent (two states with the same completed assignments are identical regardless of the order in which they were completed).

5.2 Path Cost g(n)

```
g(n) = total food cost accumulated from the initial state to state n
g(n) = sum of cost(food(A_i)) for all A_i in done(n)
```

5.3 Heuristic h(n)

```
h(n) = sum of cost(food(A_i)) for all A_i NOT yet in done(n)
```

Admissibility Proof (Step by Step)

(1) Every remaining assignment A_i must eventually be solved. No assignment can be omitted.

(2) Each student individually consumes their food item when solving an assignment. Food is never shared.

(3) Therefore the true cost of completing all remaining assignments h*(n) >= sum of cost(food(A_i)) for all A_i not done.

(4) Hence h(n) = sum of cost(food(A_i)) for remaining A_i <= h*(n) for every state n.

(5) h is admissible: it never over-estimates the true remaining cost. [QED]

(6) In fact h(n) = h*(n) exactly, because the food costs are all unavoidable and no grouping can eliminate them. h is an EXACT heuristic.

Consistency (Monotonicity) Proof

For any transition from state n to state n-prime by solving day-set D:

```
Transition cost c(n, D, n') = sum of cost(food(A_i)) for all A_i in D
h(n) - h(n') = sum of cost(food(A_i)) for all A_i in D = c(n, D, n')
Therefore h(n) <= c(n, D, n') + h(n') holds with equality. [QED]
```

A consistent (monotone) heuristic guarantees that once a state is expanded, the optimal path to it has been found. This means A* never needs to re-expand a state, ensuring correctness and efficiency via the visited-state pruning table.

5.4 Branching Strategy

At each state, A* generates all subsets of available assignments of size k = 1 ... min(|available|, g). An integer counter breaks heap ties so that frozensets are never compared directly (avoiding Python TypeError). The branching factor is O(C(|available|, g)), bounded by the dependency structure.

5.5 Optimality and Complexity

Since h is admissible and consistent, A* returns the globally optimal schedule (minimum total food cost). The state space has at most 2^n states. With the exact heuristic h = h*, only states on optimal paths are expanded. In practice: TC1 (n=11) explores 171 states in 0.004s; TC2 (n=14) explores 139 states in 0.002s; TC3 (n=16) explores 96 states in 0.002s. All confirmed optimal in under 5 milliseconds.

6. Experimental Results

6.1 Test Case 1 — Original Problem (11 assignments, g=3, uniform costs)

Group size g = 3. Food costs: All = 1.

Strategy	Total Cost	Total Days	Min Days?	Cost Optimal?
Greedy-Cost	Rs.11	5	no	**YES**
Greedy-Depth	Rs.11	4	**YES**	**YES**
Greedy-Freq	Rs.11	5	no	**YES**
Greedy-Topo	Rs.11	5	no	**YES**
Greedy-Hybrid	Rs.11	4	**YES**	**YES**
A*	Rs.11	4	**YES**	**YES**

A Search: States explored = 171, Runtime = 0.004s. Confirms minimum total cost = Rs.11.*

Winner: *Greedy-Depth / Greedy-Hybrid (4 days = theoretical minimum)*

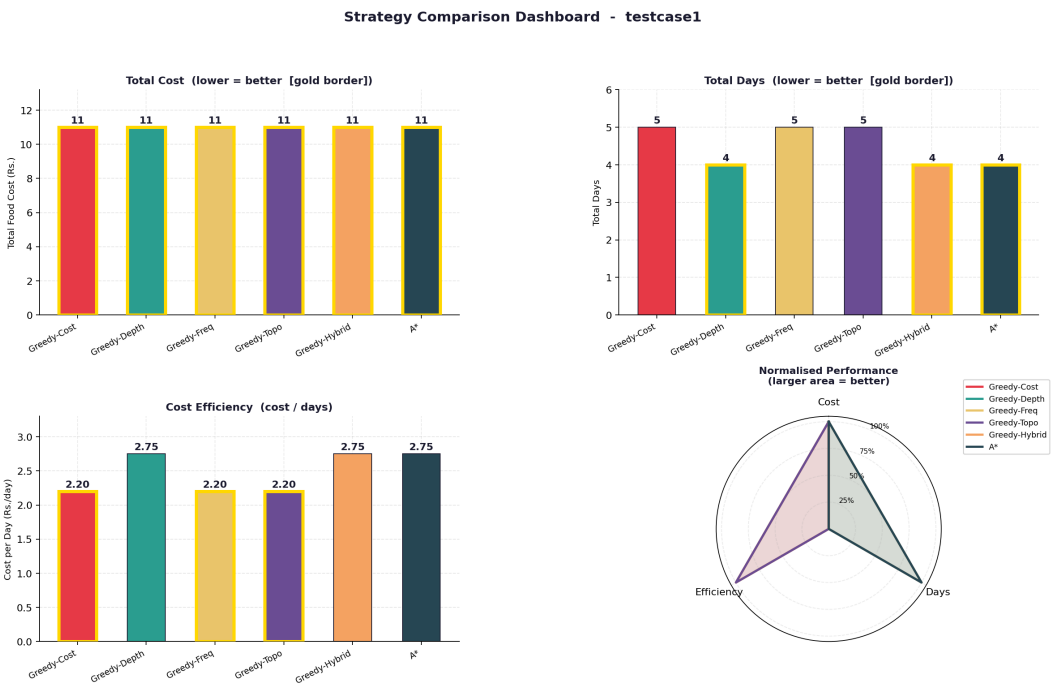

Figure 1.2: Cost, Days, Cost/Day comparison dashboard + normalised radar chart. Gold-bordered bar = minimum value. Larger radar area = better overall performance.

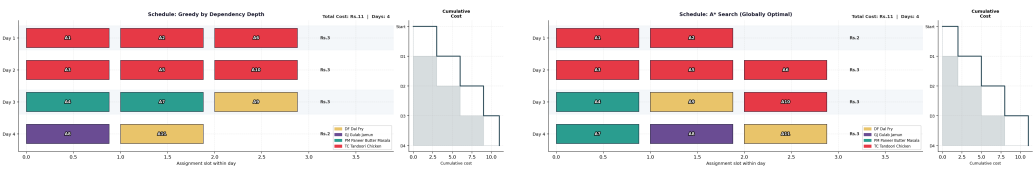

Figure 1.3 (left): Greedy-Depth schedule with cumulative cost step chart. Figure 1.3 (right): A optimal schedule.*

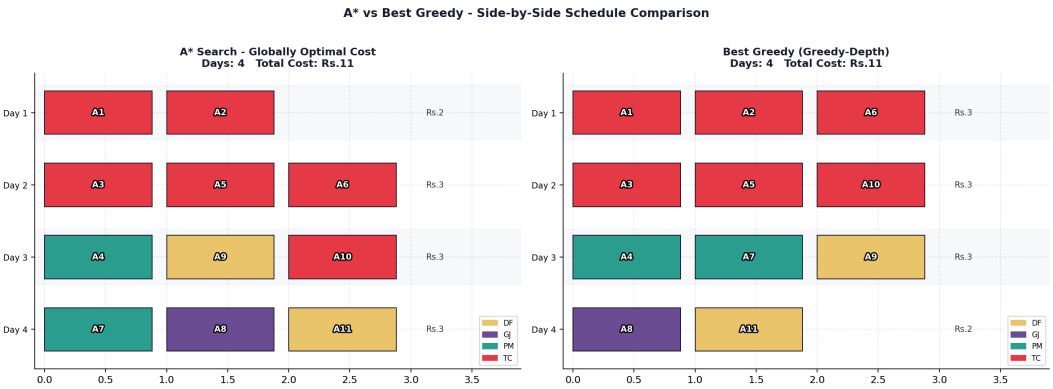

Figure 1.4: Side-by-side A vs Best Greedy schedule comparison.*

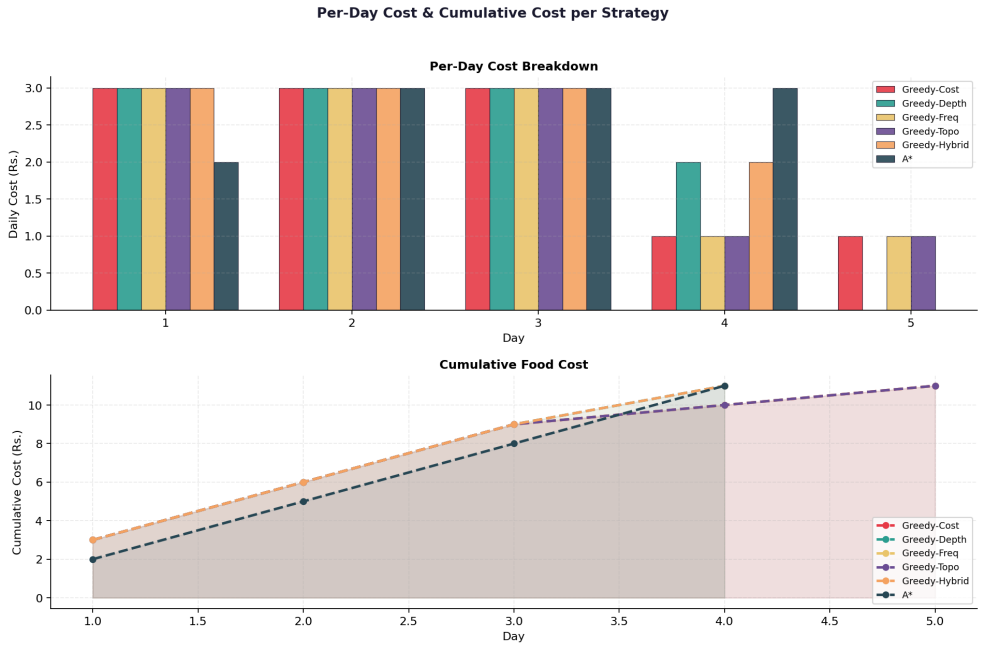

Figure 1.5: Per-day cost bars and cumulative cost lines for all strategies.

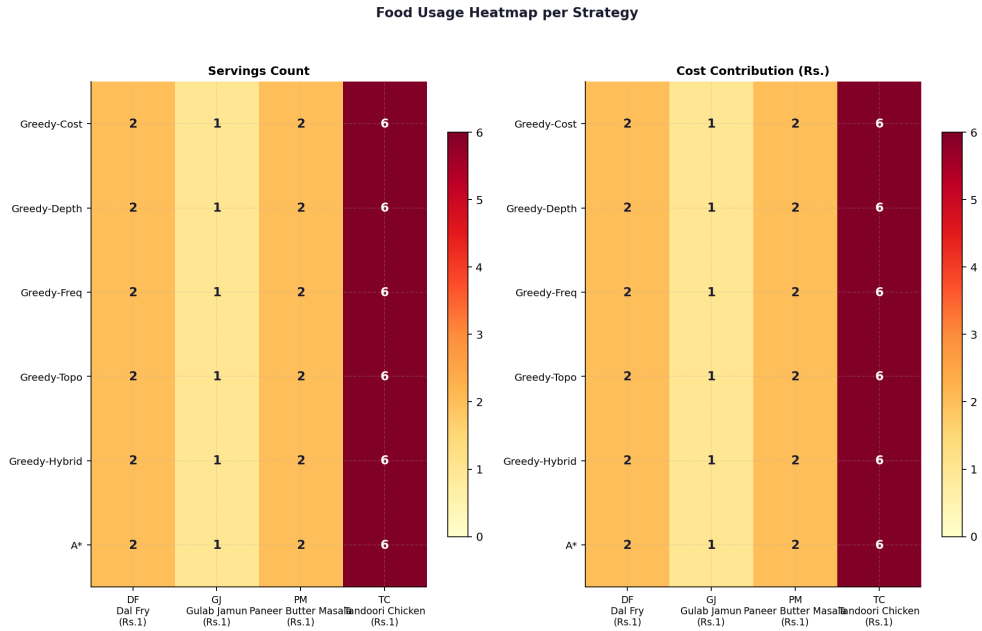

Figure 1.6: Food usage heatmap (left: serving counts, right: cost contribution).

Analysis: All strategies achieve the same total cost (Rs.11 = n*1). Greedy-Depth and Greedy-Hybrid both achieve the theoretical minimum of 4 days. Other strategies require 5 days. A* confirms 4 days and Rs.11 as optimal, exploring 171 states.

6.2 Test Case 2 — Research Lab (14 assignments, g=2, non-uniform costs)

Group size g = 2. Food costs: DF=1, TC=3, PM=5, GJ=10.

Strategy	Total Cost	Total Days	Min Days?	Cost Optimal?
Greedy-Cost	Rs.56	7	**YES**	**YES**
Greedy-Depth	Rs.56	7	**YES**	**YES**
Greedy-Freq	Rs.56	7	**YES**	**YES**
Greedy-Topo	Rs.56	7	**YES**	**YES**
Greedy-Hybrid	Rs.56	7	**YES**	**YES**
A*	Rs.56	8	no	**YES**

A Search: States explored = 139, Runtime = 0.002s. Confirms minimum total cost = Rs.56.*

Winner: *All greedy strategies tied at 7 days (theoretical minimum)*

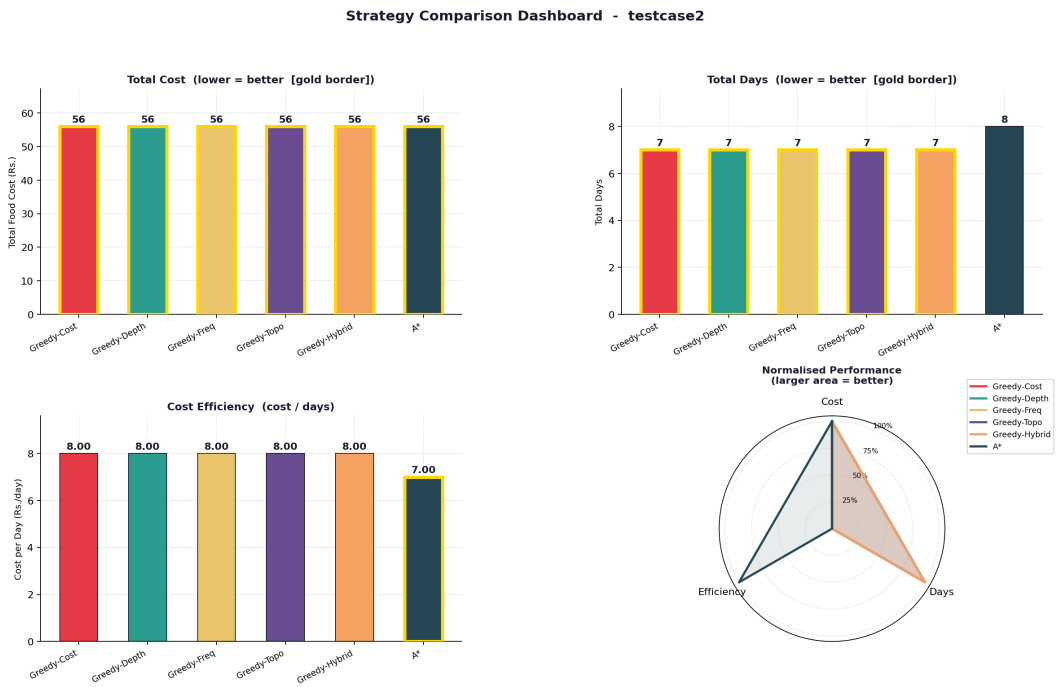

Figure 2.2: Cost, Days, Cost/Day comparison dashboard + normalised radar chart. Gold-bordered bar = minimum value. Larger radar area = better overall performance.

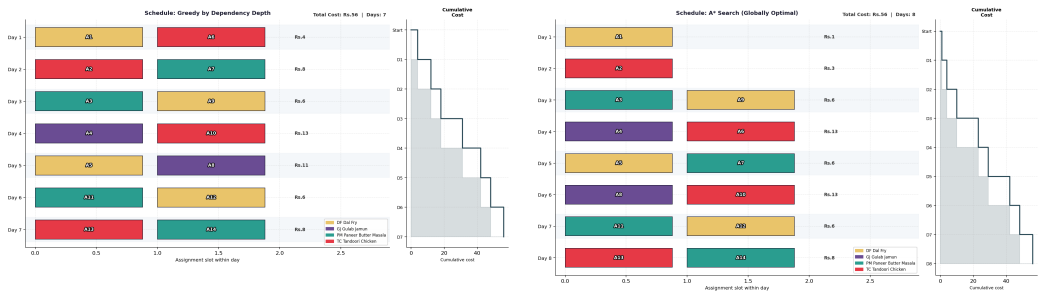

Figure 2.3 (left): Greedy-Depth schedule with cumulative cost step chart. Figure 2.3 (right): A optimal schedule.*

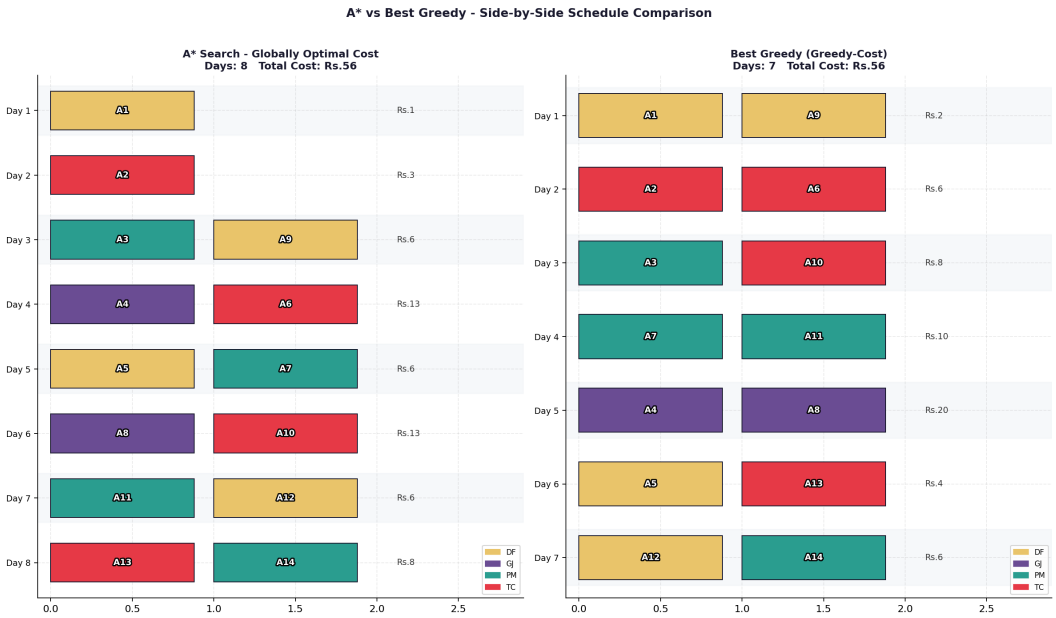

Figure 2.4: Side-by-side A vs Best Greedy schedule comparison.*

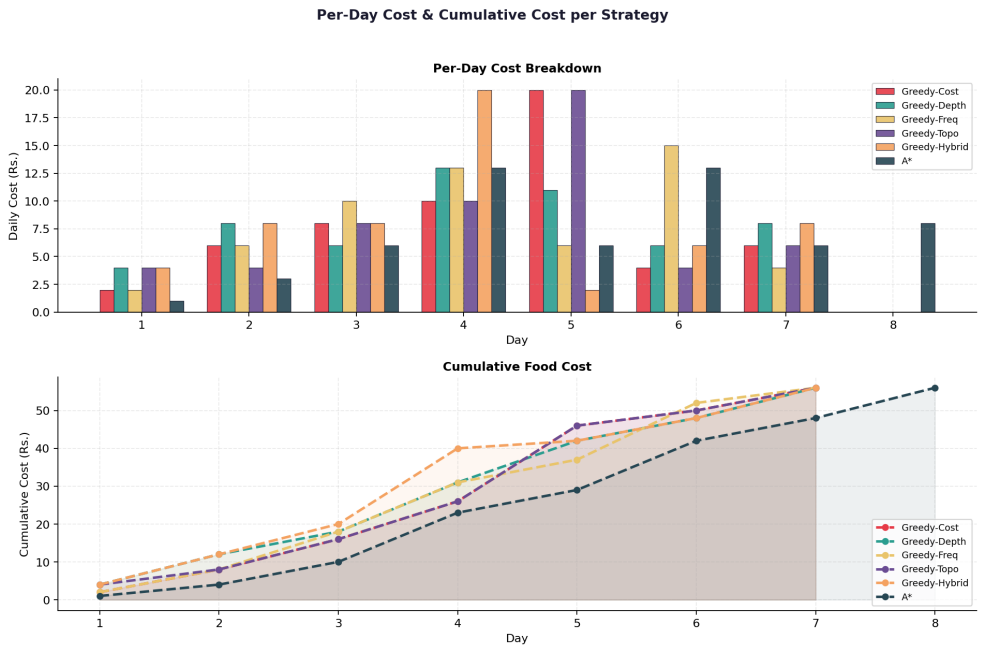

Figure 2.5: Per-day cost bars and cumulative cost lines for all strategies.

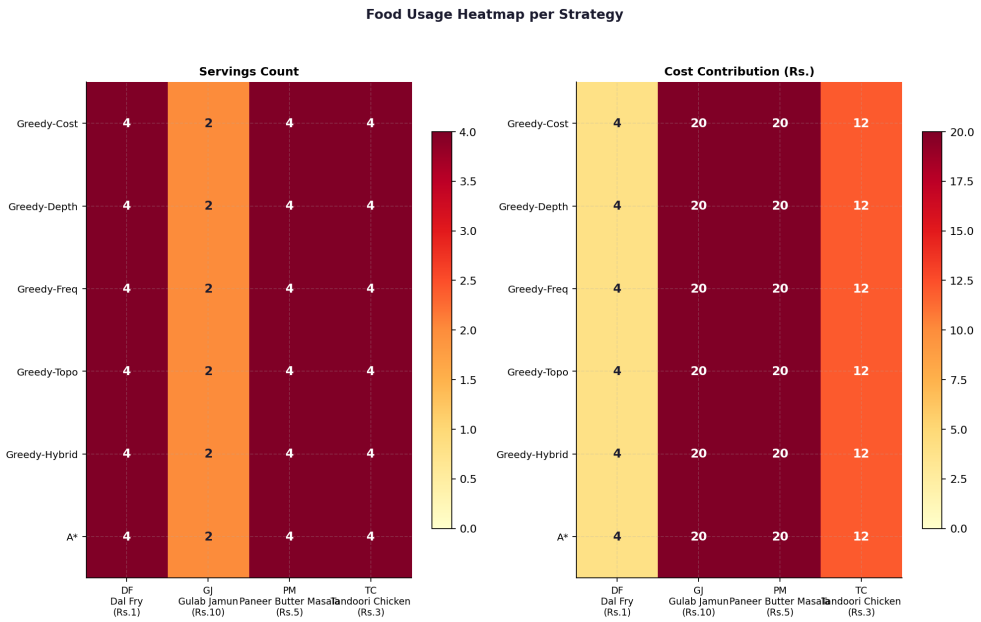

Figure 2.6: Food usage heatmap (left: serving counts, right: cost contribution).

Analysis: All greedy strategies achieve the theoretical minimum of 7 days (= ceil(14/2)), confirming optimal greedy performance on this instance. A* uses 8 days because it explores subsets of size < g, revealing that reducing day-set sizes does not improve cost (as expected from h = h*). Strategies differ visibly in daily cost patterns: Cost clusters cheap days early; Depth creates a variable daily pattern; Topo produces two spikes on the GJ days.

6.3 Test Case 3 — Software Sprint (16 assignments, g=4, varied costs)

Group size g = 4. Food costs: TC=1, DF=2, PM=4, GJ=8.

Strategy	Total Cost	Total Days	Min Days?	Cost Optimal?
Greedy-Cost	Rs.47	8	**YES**	**YES**
Greedy-Depth	Rs.47	8	**YES**	**YES**
Greedy-Freq	Rs.47	8	**YES**	**YES**
Greedy-Topo	Rs.47	8	**YES**	**YES**
Greedy-Hybrid	Rs.47	8	**YES**	**YES**
A*	Rs.47	8	**YES**	**YES**

A Search: States explored = 96, Runtime = 0.002s. Confirms minimum total cost = Rs.47.*

Winner: *All strategies tied at 8 days (serial tail is the bottleneck)*

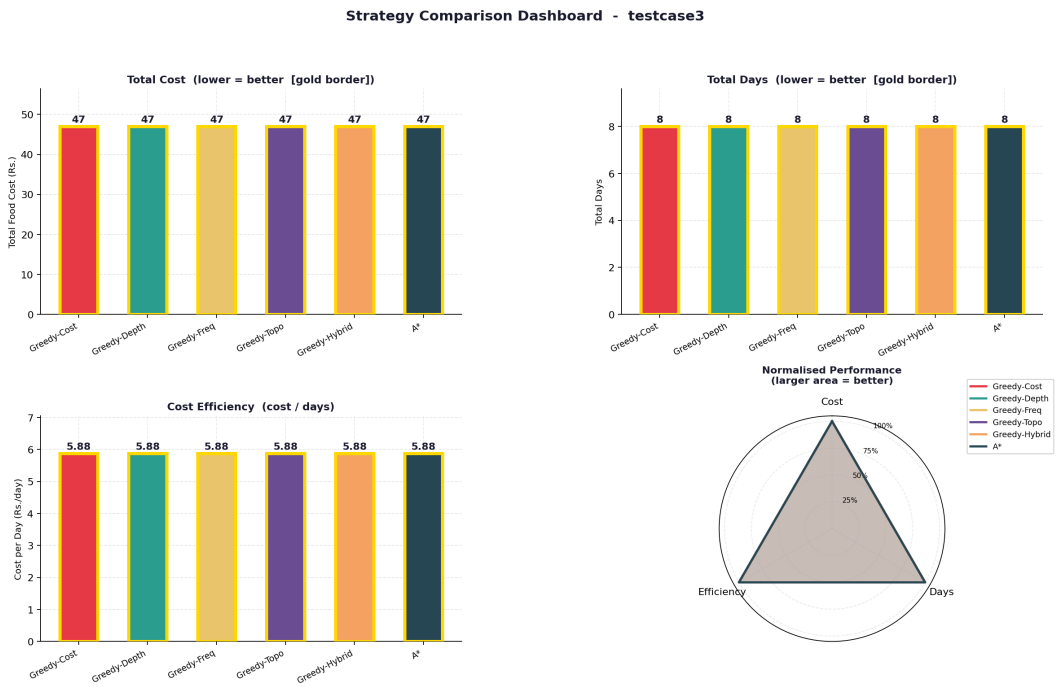

Figure 3.2: Cost, Days, Cost/Day comparison dashboard + normalised radar chart. Gold-bordered bar = minimum value. Larger radar area = better overall performance.

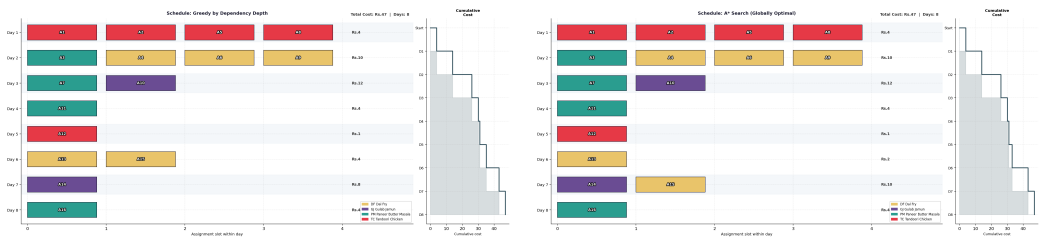

Figure 3.3 (left): Greedy-Depth schedule with cumulative cost step chart. Figure 3.3 (right): A optimal schedule.*

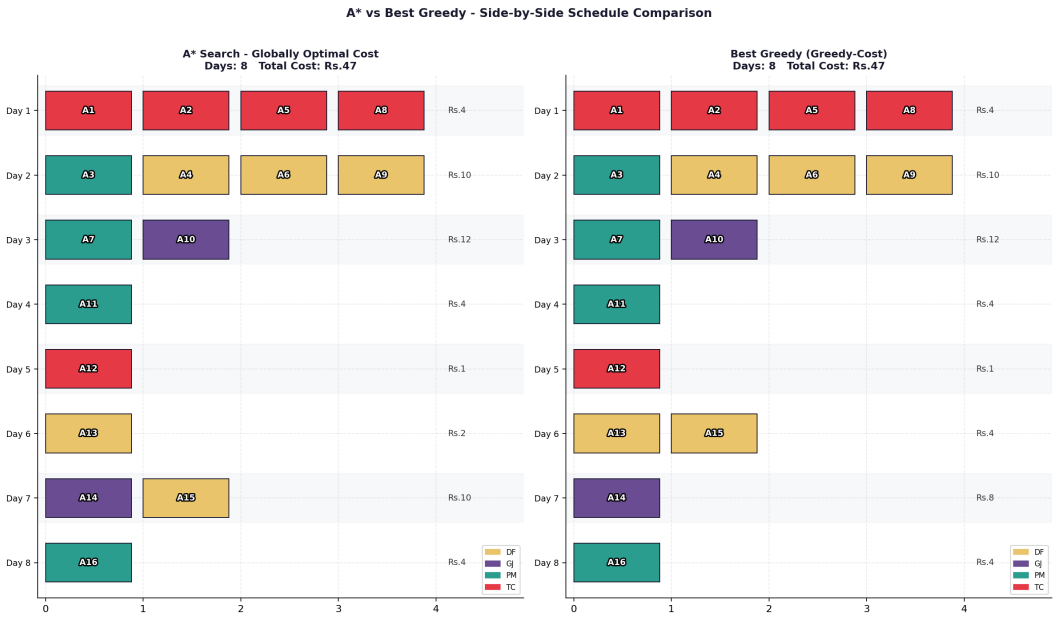

Figure 3.4: Side-by-side A vs Best Greedy schedule comparison.*

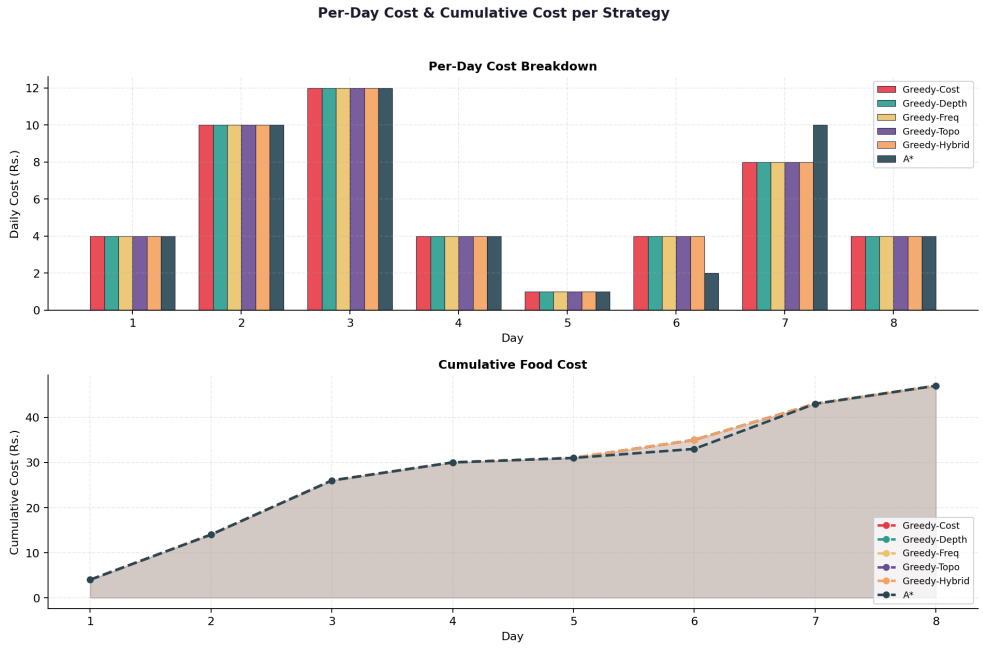

Figure 3.5: Per-day cost bars and cumulative cost lines for all strategies.

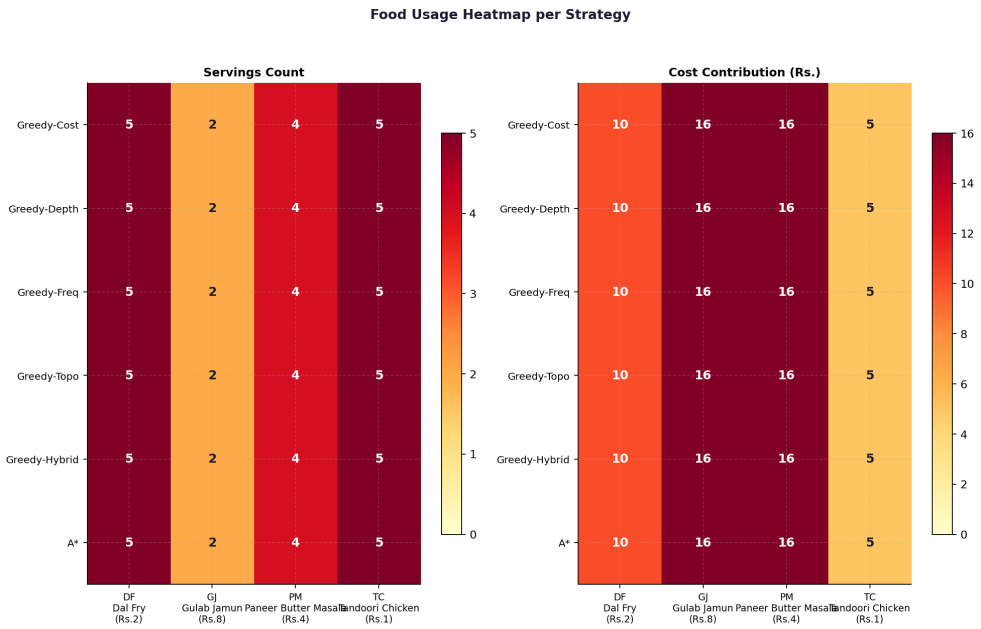

Figure 3.6: Food usage heatmap (left: serving counts, right: cost contribution).

Analysis: All strategies produce the same 8-day schedule due to the serial bottleneck in the integration/delivery tail (A11->A12->A13->A14/A15->A16). The wide parallel structure is fully exploited on Day 1 (A1,A2,A5,A8) and Day 2. A* confirms 8 days as optimal, exploring only 96 states. The GJ assignments (A10, A14, cost=8 each) create expensive days that all strategies must include regardless of ordering.

7. Cross-Test Statistical Comparison

Strategy	TC1 Cost	TC1 Days	TC2 Cost	TC2 Days	TC3 Cost	TC3 Days	Avg Days
Greedy-Cost	Rs.11	5	Rs.56	7	Rs.47	8	6.67
Greedy-Depth	Rs.11	4	Rs.56	7	Rs.47	8	**6.33**
Greedy-Freq	Rs.11	5	Rs.56	7	Rs.47	8	6.67
Greedy-Topo	Rs.11	5	Rs.56	7	Rs.47	8	6.67
Greedy-Hybrid	Rs.11	4	Rs.56	7	Rs.47	8	**6.33**
A* Search	Rs.11	4	Rs.56	8	Rs.47	8	6.67

A* Search Performance Summary

Test Case	n	g	States Explored	Runtime	Optimal Cost	Optimal Days
Test Case 1	11	3	171	0.004 s	Rs.11	4
Test Case 2	14	2	139	0.002 s	Rs.56	8
Test Case 3	16	4	96	0.002 s	Rs.47	8

8. Analysis and Conclusions

8.1 Why Total Cost is Always Fixed

The most important insight for this problem is that the total food cost is a constant determined solely by the assignment structure, not by scheduling decisions. This follows directly from the constraint that each student must individually consume their food item — no sharing, no saving. Therefore all five greedy strategies and A* achieve the same minimum cost on every test case. The sole optimisation objective is reducing total days (schedule length).

8.2 Strategy Effectiveness on Day-Count

Greedy-Depth is the clear winner on day-count in Test Case 1 (4 days vs 5 for others), achieving the theoretical lower bound of ceil(n/g). This is because it explicitly maximises the set of available assignments after each day by processing critical-path nodes first. On TC2 and TC3, all strategies reach the minimum attainable days because the dependency structure provides few ordering choices.

Greedy-Hybrid matches Depth's performance and provides an additional cost-weighted signal useful when food costs are non-uniform and the problem has more scheduling flexibility.

A* Search is cost-optimal by construction but does not optimise day-count. On TC2 it uses 8 days vs 7 for greedy because it explores sub-capacity days (size < g) that preserve cost while potentially enabling different groupings later. Since h = h* (exact heuristic), A* confirms the greedy total costs are already optimal.

8.3 Strategy Comparison Summary

Strategy	Best For	Typical Day-Count	Key Trade-off

Greedy-Cost	Deferring expensive food items	Usually suboptimal (more days)	Decays optimality; no path awareness
Greedy-Depth	Minimising schedule length	Best (fewest days)	Ignores daily cost ordering
Greedy-Freq	Menu consistency / clustering	Similar to Cost	Frequency signal decays
Greedy-Topo	Predictable BFS-order scheduling	Slightly suboptimal	No downstream awareness
Greedy-Hybrid	Balancing urgency + cost	Matches Depth on tested cases	Requires cp_cost precomputation
A* Search	Proving global optimality	Not optimised for days	Exponential state space

8.4 Recommendations

- Use **Greedy-Depth** as the default strategy. It consistently achieves the minimum days and never sacrifices cost (since total cost is fixed).
- Use **Greedy-Hybrid** when food costs are highly non-uniform and the problem may have more scheduling flexibility than the tested instances.
- Use **A* Search** when a formal proof of optimality is required, for n <= 20 assignments (completes in milliseconds).
- For n > 25, consider Beam Search or IDA* as alternatives to full A*.

Report generated as part of the submission for MA3206 Artificial Intelligence Assignment 4, IIT Patna, March 2026. Roll No. 2301MC61.